

MIDISTRUCT — Full Technical Description

Algorithmic MIDI Composer for REAPER

Developed by Acrosonus Mastering — acrosonus@outlook.com

License: GNU GPL v3 — <https://www.gnu.org/licenses/gpl-3.0.html>

Overview

MIDISTRUCT is a ReaScript Lua script for complete algorithmic composition in REAPER. From a chord progression and a musical style, it generates in a few seconds a structured 3–4 minute arrangement across 5 separate MIDI tracks, with color-coded regions, named items, CC automation and a text report. This is not an arpeggiator or a random pattern generator. Every musical decision is grounded in real compositional rules, calibrated against the note-by-note analysis of 25 pop/R&B/rock hit MIDI files.

What MidiStuct creates automatically in REAPER

- 5 color-coded MIDI tracks: DRUMS, BASS, PAD-CHORDS, MELODY, COUNTER-MEL
 - Named MIDI items per section on every track
 - Color-coded timeline regions (Intro, Verse, Chorus 1/2/3, Bridge, Outro...)
 - CC#11 Expression automation on Pad (exponential swell)
 - CC#1 Modulation automation on Melody (vibrato on long notes)
 - Tempo automation (Bridge -2 BPM, final Chorus +1 BPM)
 - Text report with seed, key, structure and hook description
-

Code Architecture

The script is organized into strictly separated layers:

RNG → Deterministic pseudo-random generator (LCG) Humanizer → Timing, velocity, rubato, push & pull Engine → Full musical engine REAPER layer → MIDI writing, regions, CC, report

No mutable global variables shared between layers.

The Engine is instantiated once and carries all internal state.



Single Lua file, no external dependencies.

1. RNG — Linear Congruential Generator (LCG)

Problem solved

Standard Lua uses `math.random` which relies on the platform's C RNG — not reproducible across sessions. A self-contained, deterministic RNG with no external dependencies was required.

Solution: Lehmer LCG (ANSI C constants)

```
seed_n+1 = (seed_n × 1103515245 + 12345) mod 2^31  
rand() = seed_n+1 / 2^31 ∈ [0, 1)
```

Period = $2^{31} = 2,147,483,648$. Same seed → identical result guaranteed, cross-platform, cross-session.

Gaussian distribution — Box-Muller transform

```
Z = sqrt(-2 × ln(U1)) × cos(2π × U2)
```

Applied to velocity, timing and rubato to produce statistically realistic micro-variations around a central value with a configurable standard deviation.

Weighted selection — Roulette wheel

`RNG:choices(seq, weights)` accumulates weights until a random threshold is crossed.

Used wherever musical choices have differing probabilities.

Seed fix (V4)

When the user enters `0`, the seed is computed via `reaper.time_precise() * 1000 mod 2147483647`

before instantiating the Engine, so it is always logged in the report and confirmation dialog. The report never displays `0` again.

2. Humanizer — Modeling Human Performance

Timing: Gaussian noise controlled by tightness

```
σ = 5.5 / tightnesstiming = floor(base_tick + gauss(0, σ))
```

tightness: 0.6 (Lo-Fi, very loose) → 2.0 (Techno, near-quantized).

Rubato: systematic offset by step position



Step 15 (phrase end) → $\text{gauss}(-6, 2.0) \times \text{intensity}$ (strong anticipation)
Step 7 (mid-bar) → $\text{gauss}(-3, 1.5) \times \text{intensity}$ (light anticipation)
Step ×4 (strong beats) → $\text{gauss}(+2, 1.0) \times \text{intensity}$ (natural delay)

This asymmetry reproduces the natural behavior of a musician who anticipates phrase endings and leans slightly into strong beats.

Push & Pull — Global offset per section

Chorus : -12 ticks (push — urgency, energy)
PreChorus : -6 ticks (light push)
Verse : +10 ticks (pull — laid-back, relaxed)
Bridge : +8 ticks (light pull)

Applied to all melody and counter-melody notes in the section.

Technique consciously used by R&B and pop producers since the 1980s.

3. Key Inference — Pitch Class Voting

Algorithm

Every pitch class (0–11) is tallied across all chords in the progression.

Chord roots are weighted double (weight 2 vs 1 for other notes).

Global root = pitch class with the highest score.

Mode is determined by the presence of the minor third (root+3):

Pop → major by default when ambiguous
Trap, House, Lo-Fi → minor by default
R&B → Dorian (minor with major sixth)
Techno → Phrygian (minor with minor second)

Global scale

Built across octaves 3 to 6 (MIDI 48–84), deduplicated, sorted.

All melody notes come from this scale — except intentional chromatic passing notes which are outside it by definition.

4. Voice Leading — Minimizing Voice Movement

For pads, each voice moves as little as possible between chords.

for each pc in chord_pcs: $\text{best_note} = \text{argmin}_{\{\text{oct} \in [4,6]\}} |\text{pc} + \text{oct} \times 12 - \text{prev_voice}[i]|$

Maximum spacing constraint of 14 semitones between adjacent voices

to prevent non-idiomatic open voicings.

5. Hook — Signature Intervals



Intervals by style

Style	Intervals	Character
Pop	7, 9, 5	Fifth, Maj sixth, Fourth
Lo-Fi	3, 5, 7	Min third, Fourth, Fifth
R&B	9, 7, 12	Maj sixth, Fifth, Octave
Techno	1, 6, 11	Min second, Tritone, Maj seventh
Rock	7, 5, 10	Fifth, Fourth, Min seventh
Trap	3, 10, 7	Min third, Min seventh, Fifth

Chromatic intervals are converted to scale degrees via an `iv_to_deg` mapping table, allowing intervallic contours that remain within the scale regardless of key.

Rhythmic patterns with intentional syncopation

6 rhythmic patterns available, with deliberately placed syncopation steps

(steps 3, 7, 11 — a sixteenth note before the strong beats).

These steps have a 1.15× higher probability of being played.

6. ×3 Repetition Rule — Inter-section

State machine



`_hook_play_count` -- times played in current variation `_hook_variation` -- current level (0=original, 1=+1deg, 2=+oct, 3=+oct+harm)

The hook repeats 3 times identically, then advances to the next level.

The 4 variation levels:

- **0**: original hook
- **1**: transposed +1 scale degree
- **2**: transposed +1 octave
- **3**: octave + third harmonization on even beats

7. Intra-phrase Repetition

Principle (the most important musical gap closed)

In a real hit, bars 1 and 2 of a 4-bar group are often **identical**.

Variation only arrives in bar 3. This is what creates memorability.

Cache implementation

`phrase_role "A"` → generates normally, saves to `_phrase_cache`
`phrase_role "Ap"` → returns exact copy of cache (bar 2 = bar 1)
`phrase_role "A2"` → returns cache with light variation + resolution note
`phrase_role "B"` → contrasting phrase (lower register, long notes)

Cache key is `section + chorus_index` to distinguish Chorus 1/2/3.

```
-- Ap: exact copy
if phrase_role=="Ap" and self._phrase_cache[cache_key] then
  for s, m in pairs(self._phrase_cache[cache_key]) do
    melody[s] = {note=m.note, dur=m.dur, ...}
  end
  return melody
end
-- A2: light variation (55% chance of shifting 1 degree toward resolution)
if phrase_role=="A2" and self._phrase_cache[cache_key] then
  for s, m in pairs(cache) do
    if not m.is_motif and rng:rand()<0.55 then
      dir = (note < target_note) and 2 or -2
      note = nearest_scale_note(note+dir)
    end
    melody[s] = {note=target_note, is_resolution=true}
  end
  return melody
end
```

8. Role-driven Note Durations

Problem solved

Random durations `choices({1,2,3,4}, {0.25,0.40,0.25,0.10})` ignored the

musical role of each note. Natural phrasing has durations consistent with

the function of each note.

role_dur function

```
local function role_dur(is_pass, is_strong, is_resol, is_mot)
  if is_pass then return 1
  end
  -- Fixed sixteenth
  if is_resol then return choices({3,4},{0.55,0.45})
  end
  -- Half/whole note
  if is_mot then return choices({2,3},{0.65,0.35})
  end
  -- Dotted
```



```
quarter    if is_strong then return choices({2,3},{0.70,0.30}) end -- Quarter note    return
choices({1,2},{0.40,0.60})                -- Eighth noteend
```

Applied to:

- All chromatic passing notes (is_pass=true → 1 step, fixed)
- All hook notes (is_mot=true → 2-3 steps)
- All climax and strong-beat notes (is_strong=true)
- Phrase-ending resolution notes (is_resol=true → 3-4 steps)

9. Melodic Structure A-A'-B-A

Contours per section

Section	Contour	Description
Intro	flat	Stable
Verse	wave	$\sin(s \times 0.8) \times 3$
PreChorus	rise	Linear ascent
Chorus	peak	Parabolic arc
Bridge	fall	Descent
Outro	fall	Resolution

Single climax

Each Chorus and PreChorus forces a climax note at step 10:

```
climax = nearest_scale_note(target_note + signature_interval[1])vel_boost = max(0, 1.0 - |step - 10| × 0.08)
```

Notes near the climax are up to 25% louder.

Chromatic passing notes

On odd steps: 20% probability of inserting an out-of-scale note

(±1 semitone from the previous note). Duration fixed to 1 step by `role_dur`.



10. Section-differentiated Walking Bass

Problem solved

In a real pop/R&B arrangement, the bass is sparse in the Verse (leaves space), picks up in the PreChorus, and is full in the Chorus.

Three levels

Sparse (Verse, Bridge, Intro, Outro):

Root on beats 1 and 3 only + chromatic approach note → Lots of silence, melody breathes

Intermediate (PreChorus):

Root + fifth, growing density → Builds toward the Chorus drop

Full (Chorus + active sections):

Complete pattern (root, fifth, third, seventh, approach) + High octave (+12 MIDI) → Maximum energy

Call & response (adding notes in melody silences) is only active on non-sparse sections.

11. Inter-track Call & Response

Shared density map

`density_map[0..15] = 0 or 1`

`_gen_melody` marks occupied steps as 1.

`_gen_counter_melody` preferentially targets steps marked 0 (melody silences).

`_gen_bass` enriches silences on odd bars (25% probability).

12. Secondary Dominants

Before chord changes with root distance ≥ 3 semitones:

`sec_dom_root = (next_chord_root + 7) mod 12`

Inserted with 35% probability as a short passing chord (type "7").

Creates local harmonic resolutions that smooth transitions.

13. CC Automation — Exponential Swell and Vibrato

CC#11 Expression — First-order step response (pad)

One CC#11 point every 60 PPQ (granularity: sixteenth note):

Attack phase (65% of duration):



$t = \text{step_index} / \text{attack_steps} \in [0, 1]$
 $\text{exp_val} = 1 - e^{-3t}$
 $\text{val} = \text{start} + (\text{peak} - \text{start}) \times \text{exp_val}$

Models the response of a first-order system — identical to the physical behavior of a string swell or analog pad.

Release phase (35% of duration): linear descent to 82% of peak.

CC#1 Modulation — Vibrato envelope

On melody notes ≥ 2 steps (480 PPQ):

$t = \text{ppq} + \text{dur}/3 \rightarrow \text{CC\#1} = 0$ (vibrato onset)
 $t = \text{ppq} + 2 \times \text{dur}/3 \rightarrow \text{CC\#1} = 18$ (fade out)
 (Chorus) or 10 (other)
 $t = \text{note_end} - 20 \rightarrow \text{CC\#1} = 0$ (fade out)

Vibrato only starts after 1/3 of the note duration —

natural behavior of a singer or string player.

REAPER API call (corrected V4)

```
reaper.MIDI_InsertCC(take, false, false, ppq, 0xB0, 0, 11, val, false)--
^   ^   ^   ^   ^
chanmsg chan CC# val
```

14. Drums — Open/Closed Hi-Hat + Ride Cymbal

GM notes used

Note	Instrument t	Duration	Context
36	Kick	110 ticks	Style kick patterns
38	Snare	115 ticks	Beats 2&4, ghost notes, fills
42	Closed hi-hat	95 ticks	Steps outside open_steps
46	Open hi-hat	220 ticks	Style-defined open_steps
49	Crash	800 ticks	Start of each



			section
51	Ride	95 ticks	Lo-Fi and Lo-Fi Bridge
53	Ride bell	95 ticks	Lo-Fi, beats 1 and 3

Open_steps by style

Style	Open steps	Character
Pop	4, 12	Opens on beats 2 and 4
House	8	Mid-bar
Rock	0,4,8,12	Always open (live feel)
Trap	none	Always closed

15. Melodic Density — Calibrated on MIDI Analysis

Analysis of 50 reference hits

After note-by-note MIDI analysis of reference files

(Titanium, Numb, Shake It Off, Blinding Lights,...):

- Pop lead melody: **3 to 5 notes per bar**
- R&B lead melody: **5 to 7 notes per bar**

Calibrated formula

$$\text{base_density} = 0.13 + \text{comp} \times 0.036 + (\text{is_chorus} ? 0.08 : 0)$$

Complexity	Density	Notes/bar approx
------------	---------	------------------



3-4	24-28%	~3-4 (Taylor Swift, Billie Eilish)
5	31%	~4-5 (Pop mid- tempo, EDM)
6-7	35-39%	~5-6 (R&B, Rock)
8-10	42-49%	~6-7 (Neo- Soul, Jazz)

16. REAPER Integration

Track colors (BGR + flag 0x1000000)

```
color = R + G×256 + B×65536 + 0x1000000reaper.SetMediaTrackInfo_Value(track, "I_CUSTOMCOLOR", color)
```

Color-coded regions (isrgn=true)

Change detection via composite key `section + chorus_index`

to distinguish Chorus 1, Chorus 2, Chorus 3.

```
reaper.AddProjectMarker2(0, true, region_start, region_end, label, -1, color)
```

Named items per section

```
reaper.GetSetMediaItemTakeInfo_String(take, "P_NAME", "MELODY | Chorus 2", true)
```

MIDI_DisableSort / MIDI_Sort

All insertions after `MIDI_DisableSort(take)`.

`MIDI_Sort(take)` called once per take at the end.

Divides write time by a factor proportional to the number of notes.

17. Generated Song Structure

```
Intro    (strip_level 2: drums + pad only)Verse    × 2
PreChorus × 1Chorus 1 × 2 (original hook)Verse    × 2PreChorus × 1Chorus 2 × 2
```



ACROSONUS

(hook +1 octave)Bridge × 1 (modulated +2 semitones, stripped arrangement)Chorus
3 × 2 (hook +octave + third harmonization)Outro × 1 (strip-down)

Phrase roles within each 4-bar group: A → Ap → B → A2

18. User Parameters

Dialog 1 — Style

- Primary style (1-7: Pop, Techno, Lo-Fi, R&B, House, Trap, Rock)
- Secondary style for hybrid mix (0 = none)
- Mix % (linear interpolation of numeric parameters)
- Complexity 1-10 (melodic density)
- Seed (0 = random, computed before Engine:new and always logged)

Dialog 2 — Preset or Manual

- 17 built-in presets
- 0 = manual entry with syntax `Note[Quality][:Duration]`

19. Typical Output

Blank REAPER session, `Am:4 F:4 C:4 G:4`, Pop style, complexity 5:

5 color-coded MIDI tracks~120 named MIDI items8 color-coded regions (Chorus 1/2/3 distinct)3 automatic tempo markersCC#11 on PAD (exponential curve per chord)CC#1 on MELODY (vibrato on long notes)MIDISTRUCT_rapport.txt in project folder

Compatibility

- REAPER 6.x and 7.x (Windows, macOS, Linux)
- No external dependencies
- Compatible with all GM and non-GM VSTi plugins

What the Algorithmic Engine Cannot Do

For full transparency — the limits that no algorithm can cross:

- **Perceptual evaluation:** the engine generates without knowing if a melody is good. That judgment is yours.
- **Emotional narrative:** a hit tells a specific human story.

No algorithm can invent that.



- **Timbre:** what makes a melody sound like Dua Lipa vs Billie Eilish

is 90% sound design and production. That is your VST and your mix. MIDISTRUCT gives you the 80%. The 20% that turns a good generation into a real track — choosing the right seed, adjusting the BPM, editing 3-4 notes in the piano roll, choosing the right sounds — that remains yours.

And that is exactly how it should be.

MIDISTRUCT — Acrosonus Mastering — GNU GPL v3

